

저는 ‘메이플스토리’ 라는 RPG 게임과, ‘Slayer the Spire2’ 라는 카드 턴제 게임 이 두 가지로 과제 진행해보겠습니다.

1. 메이플스토리의 주요 시스템과 데이터 구조

먼저, 메이플스토리의 주요 시스템과 데이터 구조부터 알아보겠습니다.

1) 계정 로그인

넥슨에 로그인하여 게임을 시작해야만 합니다. 이 계정에 넥슨 캐쉬와 메이플 포인트라는 재화가 같이 포함되어 있습니다.

2) 서버(월드)

메이플에는 다양한 서버가 존재합니다. 루나, 스카니아, 엘리시움 등 이 있습니다. 게임에 접속하게 되면 이 서버중 하나를 선택해 캐릭터 생성을하여 게임을 시작하게 되는 것입니다.

3) 직업과 스킬

서버를 선택했다면, 캐릭터를 생성해야하는데 캐릭터 생성시 직업은 반드시 하나를 선택해야 합니다. 이때 각 직업은 각각 고유의 스킬을 가지고, 이 스킬의 마스터 레벨이 존재합니다.

4) 캐릭터 시스템

캐릭터를 생성하면 각 캐릭터는 닉네임, 레벨, 경험치, 메소 등 각 고유의 관리 자원을 가지게 됩니다. 이 것들은 캐릭터 단위로 관리됩니다.

5) 아이템 시스템

캐릭터는 인벤토리에 여러 아이템을 보관할 수 있는데, 각 카테고리마다 분류하게 되어 있습니다. 장비, 소비, 설치, 기타, 캐쉬 등 다양한 카테고리가 존재하는데, 장비 아이템중 스타포스와 큐브라는 재화 소비 콘텐츠가 있어, 세부적으로는 같은 아이템이지만 다른 능력치를 보유하게 됩니다.

6) 캐쉬 충전과 캐쉬 아이템 구매

현금으로 캐쉬를 충전해서 그 충전한 캐쉬를 통해 캐쉬샵 아이템을 구매할 수 있습니다. 이 과정들은 맨 위에서 언급드렸듯이, 넥슨 계정단위로 진행됩니다.

2. 개체, 속성, 키 도출하기

위의 시스템들을 바탕으로 해서 약 7개정도의 개체를 도출할 수 있었습니다. 계정, 캐쉬 충전 및 구매, 서버, 직업, 캐릭터, 스킬, 보유 아이템입니다.

(1) 계정

PK : maple_ID (계정 구별 번호)

속성 : nexon_ID(넥슨 아이디), nexon_password(넥슨 비밀번호), when_ID(가입일), maple_point(메이플 포인트)

(2) 캐시충전 및 구매

PK : trade_ID (거래 구별 번호)

FK : maple_ID → Account

속성 : cashtrade_type(거래 유형: 충전/구매), cash_amount(거래 금액), when_cash(거래 날짜), cashproduct_name(캐시로 구매한 아이템, 구매일 때만 입력)

(3) 서버

PK : world_ID (월드 구별 번호)

속성 : world_name(월드 이름)

(4) 직업

PK : job_ID (직업 구별 번호)

속성 : job_name(직업 이름), job_group(소속된 직업 그룹: 전사, 마법사 등)

(5) 캐릭터

PK : char_ID (캐릭터 구별 번호)

FK : maple_ID → Account, world_ID → World, job_ID → Job

속성 : nickname(캐릭터 이름), level(캐릭터 레벨), Exp(캐릭터 경험치), meso(보유한 메소 양)

(6) 스킬

PK : skill_ID (스킬 구별 번호)

FK : job_ID → Job

속성 : skill_name(스킬 이름), skill_level_max(마스터 스킬 레벨)

(7) 보유한 아이템

PK : My_item_ID (아이템 구별 번호)

FK : char_ID → Character

속성 : item_category(아이템 카테고리), item_name(아이템 이름), item_num(아이템 수량), item_starforce(스타포스 수치), item_gradenum(주문서 강화 수치), item_potential(잠재능력 단계)

3. 개체 간 관계와 카디널리티

메이플스토리의 개체 간 관계는 모두 점선으로 이루어져 있다고 말할 수 있습니다. 왜냐하면 모든 개체와 자식 개체가 자신만의 고유 식별을 기본 키로 가지고 있어서 부모의 키를 자식이 물려받을 이유가 없기 때문에 점선형태라고 말할 수 있습니다.

(1) 계정 1 : 0..N 캐릭터 (소유)

하나의 계정은 캐릭터가 없을 수도 있고 여러 개일 수도 있습니다. 캐릭터는 반드시 하나의 계정에 속해있어야 합니다.

(2) 계정 1 : 0..N 캐시충전 및 구매 (거래)

과금하지 않는 계정은 거래 내역이 0건입니다. 과금을 한다면 반드시 하나의 계정에서 이루어져야 합니다.

(3) 월드 1 : 0..N 캐릭터 (소속)

캐릭터는 한 서버에 귀속되어 있지만, 서버에 여러개의 캐릭터를 생성할 수 있습니다.

(4) 직업 1 : 0..N 캐릭터 (직업 선택)

캐릭터는 한 개의 직업만 가져야만 하지만, 하나의 직업은 여러개의 캐릭터가 선택할 수 있습니다.

(5) 직업 1 : 1..N 스킬 (보유 스킬)

모든 스킬은 한 개의 직업에 존재하고, 각 직업은 한 개 이상의 다양한 스킬을 가집니다.

(6) 캐릭터 1 : 0..N 보유한 아이템 (보유)

인벤토리는 비어 있을 수 있고, 아이템들은 반드시 하나의 캐릭터가 보유할 수 있습니다.

4. 관계형 데이터베이스 구조 (AI 작성)

위의 1번, 2번을 진행한 뒤, ERD작성을 바로 했기 때문에 이 과정은 AI에게 부탁하여 작성하게 되었습니다.

FK는 기본키, FK->테이블은 참조하는 테이블을 의미합니다.

Account(maple_ID[FK], nexon_ID, nexon_password, when_ID, maple_point)

Cash_Trade(trade_ID[FK], cash_amount, when_cash, cashproduct_name, maple_ID[FK->Account], cashtrade_type)

World(world_ID[FK], world_name)

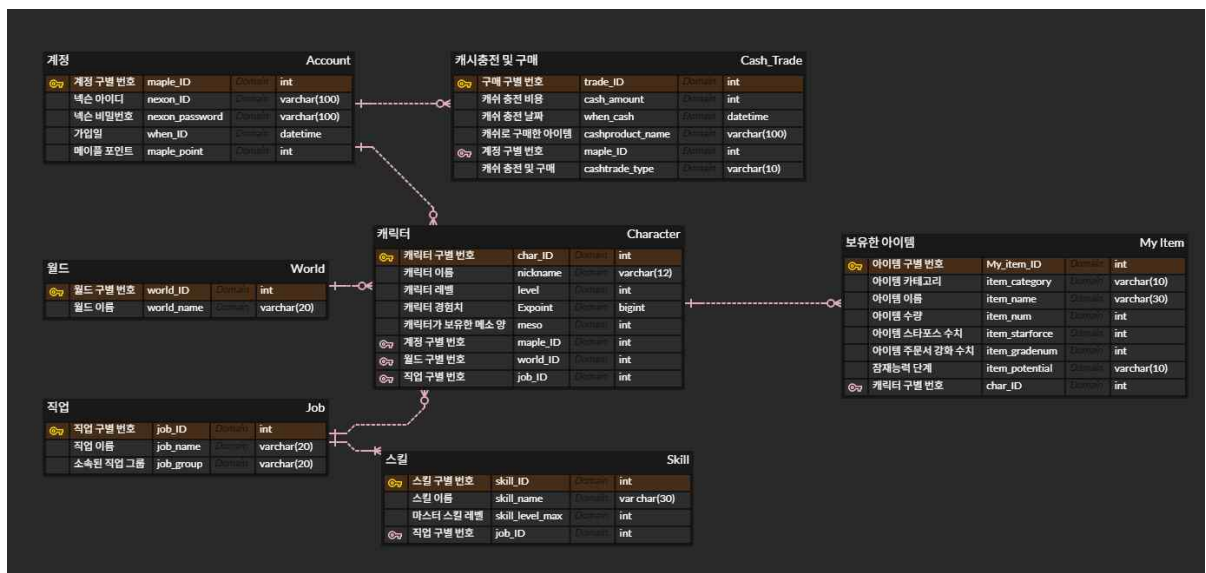
Job(job_ID[FK], job_name, job_group)

Character(char_ID[FK], nickname, level, Expoint, meso, maple_ID[FK->Account], world_ID[FK->World], job_ID[FK->Job])

Skill(skill_ID[FK], skill_name, skill_level_max, job_ID[FK->Job])

My_Item(My_item_ID[FK], item_category, item_name, item_num, item_starforce, item_gradenum, item_potential, char_ID[FK->Character])

5. ERD



ERD Cloud라는 웹페이지에서 작성했습니다.

6. 과금 데이터 구조 분석

메이플스토리 과금 구조는 계정과 캐쉬충전 및 구매 두 개의 객체로 표현 할 수 있습니다. 현금으로 충전하는 캐쉬는 계정에 귀속되고, 인게임 내의 플레이를 통해 얻을 수 있는 재화인 메소는 캐릭터에 귀속시켰습니다. 인게임에서 캐쉬는 같은 계정 내의 모든 캐릭터가 공유하는데, 메소는 캐릭터마다 따로 표시될 수 있기 때문입니다.(하지만 현재, 같은 메소를 같은 서버

내의 모든 캐릭터가 공유할 수 있는데, 메소를 다르게 가질 수도 있기에 이렇게 진행했습니다.) 캐쉬 충전과 캐쉬아이템 구매는 따로 개체를 나누지 않고 Cash_trade에 저장했고, cashtrade_type으로 충전인지 구매인지 구분하게 했습니다. 캐쉬 아이템 구매일때만 cashproduct_name을 입력하게하고, 캐쉬 충전할때는 비워두는 방식으로 진행했습니다. 예전에는 큐브와 같은 확률형 아이템들을 현금(캐쉬)로 구매해야해서 확률을 검증하기 위해 사용하는 결과를 저장해두는 개체가 필요했는데, 현재의 메이플스토리는 인게임 재화(메소)로 바뀌어 따로 추가하진 않았습니다.

1. Slayer the Spire2 의 주요 시스템과 데이터 구조

일명 슬더스2의 데이터는 플레이어 데이터, 카드나 유물같은 게임 내의 데이터, 게임을 진행하며 등반하는 데이터 이렇게 3가지로 나눌 수 있습니다. 이때, 플레이어 데이터는 등반이 끝나도 영구적으로 저장되어 되면서 누적되어야하고, 등반 데이터는 카드와 유물 데이터가 유동적으로 변해야합니다. 또, 등반이 끝나게 되면 초기화가 필수적입니다.

1) 플레이어 데이터

플레이어 정보가 누적되며 영구적으로 유지되는 데이터입니다.

2) 게임 내의 데이터

캐릭터, 카드, 유물 등 콘텐츠 데이터입니다.

3) 등반 데이터

게임을 진행하며 그때의 등반에서만 얻는 데이터입니다.

2. 개체, 속성, 키 도출

(1) 플레이어

PK : player_ID (플레이어 구별 번호)

속성 : steam_ID(스팀 아이디), total_score(누적 점수)

(2) 캐릭터

PK : character_ID (캐릭터 구별 번호)

속성 : char_name(캐릭터 이름), basic_Hp(기본 체력), basic_cardnum(기본 카드 개수)

(3) 카드

PK : card_ID (카드 구별 번호)

FK : character_ID → Player_Character (공용 카드는 NULL)

속성 : card_name(카드 이름), card_type(카드 종류), card_rare(카드 희귀도), card_energy(카드 에너지 비용)

(4) 유물

PK : relic_ID (유물 구별 번호)

FK : character_ID → Player_Character (공용 유물은 NULL)

속성 : relic_name(유물 이름), relic_rare(유물 희귀도)

(5) 게임 진행

PK : run_ID (게임 진행 구별 번호)

FK : player_ID → Player, character_ID → Player_Character

속성 : run_starttime(시작 시간), run_level(승천 단계), run_finalfloor(도달한 층),

run_result(게임 결과)

(6) 게임 내 얻은 카드

FK : run_card_ID (게임 내에서 얻은 카드 구별 번호)

FK : run_ID → Run_Game, card_ID → Card

속성 : upgrade_card(카드의 강화 상태)

(7) 게임 내 얻은 유물

FK : (run_ID, relic_ID) 복합키

FK : run_ID → Run_Game, relic_ID → Relic

속성 : gained_floor(획득한 층)

3. 개체 간 관계와 카디널리티

(1) 플레이어 1 : 0..N 게임 진행 (플레이) - 비식별 관계

게임을 막 시작한 플레이어는 런 기록이 0개일 수 있다. 각 런은 반드시 한 명의 플레이어가 진행한다.

(2) 캐릭터 1 : 0..N 게임 진행 (선택) - 비식별 관계

각 등반은 하나의 캐릭터로 진행하며, 한 번도 선택되지 않은 캐릭터도 있을 수 있습니다.

(3) 캐릭터 0..1 : 0..N 카드 (전용 카드) - 비식별 관계

캐릭터 전용 카드는 하나의 캐릭터에 속하고, 공용 카드는 모든 캐릭터가 사용할 수 있습니다. 그렇기 때문에 캐릭터 쪽 최소값이 0이 됩니다.

(4) 캐릭터 0..1 : 0..N 유물 (전용 유물) - 비식별 관계

카드와 같이 공용 유물은 귀속되어 있는 캐릭터가 없습니다.

(5) 게임 진행 1 : 1..N 게임 내 얻은 카드 - 비식별 관계

모든 등반은 시작 카드를 가지고 시작해서 반드시 최소 1장 이상의 카드를 가지게 됩니다.

(6) 카드 1 : 0..N 게임 내 얻은 카드 - 비식별 관계

등반 중에 한 번도 얻지 못한 카드가 존재할 수 있으므로, 최소값은 0입니다.

(7) 게임 진행 1 : 1..N 게임 내 얻은 유물 (보유) - 식별 관계

모든 등반에서 캐릭터는 시작 유물을 가지고 시작하므로 최소 1개 이상의 유물을 가지게 됩니다.

(8) 유물 1 : 0..N 게임 내 얻은 유물 - 식별 관계

게임 진행과 카드, 게임 진행과 유물은 다대다 관계입니다. 한번의 등반에는 여러 카드와 유물이 있고, 하나의 카드와 유물은 여러 등반 중에 등장합니다. 이 다대다 관계를 각각 게임 내 얻은 카드와 게임 내 얻은 유물 연결 개체로 해서 1:N 관계 두 개로 나누었습니다.

4. 관계형 데이터베이스 구조(AI 작성)

Player(player_ID[FK], steam_ID, total_score)

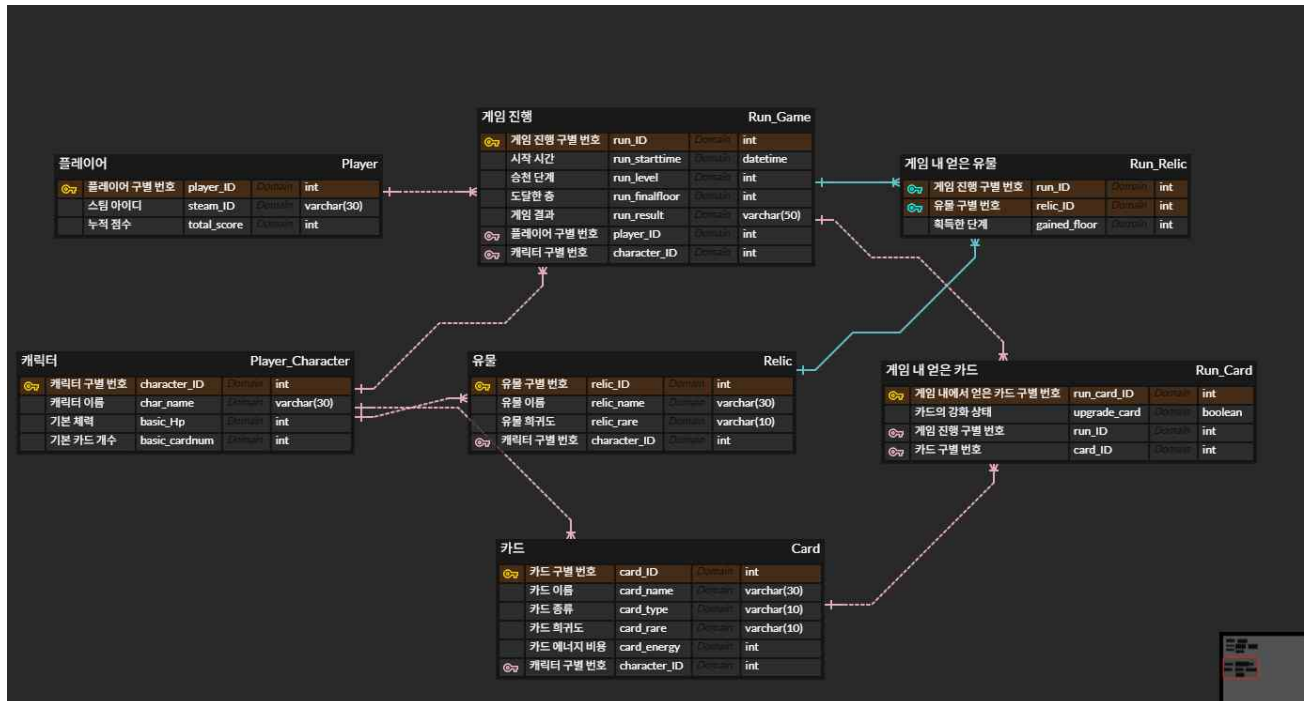
Player_Character(character_ID[FK], char_name, basic Hp, basic_cardnum)

Card(card_ID[FK], card_name, card_type, card_rare, card_energy, character_ID[FK → Player_Character, NULL 허용])

Relic(relic_ID[FK], relic_name, relic_rare, character_ID[FK → Player_Character, NULL 허용])

Run_Game(run_ID[PK], run_starttime, run_level, run_finalfloor, run_result, player_ID[FK → Player], character_ID[FK → Player_Character])
 Run_Card(run_card_ID[PK], upgrade_card, run_ID[FK → Run_Game], card_ID[FK → Card])
 Run_Relic(run_ID[FK, FK → Run_Game], relic_ID[PK, FK → Relic], gained_floor)

5. ERD



6. 과금 구조 분석

슬더스2는 스팀에서 한번 구매하면 그 후 과금이 없는 패키지 게임입니다. 따로 인게임 내에서 과금하는 시스템도 존재하지 않습니다. 등반 중에 얻는 골드는 그 등반에서만 사용 가능하고, 등반이 초기화 되면 사라지는 재화여서 현금가치와는 관계가 없습니다. 그렇기 때문에 데이터베이스에 결제나 구매와 관련된 개체가 존재할 수 없습니다. 게임 구매 기록은 게임에서가 아니라 steam 플랫폼이 관리를 해야하는 문제라 게임은 나의 steam id로 플레이어가 패키지 게임을 구매했는지만 식별하면 됩니다.

여기까지 각 메이플스토리과 슬더스2의 데이터베이스 구조를 파악해보았습니다. 이제 두 게임의 구조를 비교 분석해보겠습니다.

1) 데이터가 누적인가 일회성인가

메이플스토리는 캐릭터, 보유한 아이템, 메소가 삭제되지 않는 한 계속 누적되고, 플레이어가 들인 시간과 비용이 데이터 자체에 쌓이게 됩니다. 슬더스2는 플레이 중 가장 많이 변하는 데이터가 모두 게임 진행에 의존하며, 등반이 끝나면 더 이상 사용되지 않고 증발하게 됩니다. 게임 디자인이 데이터의 누적인지, 매번 진행할때마다 초기화인지가 데이터베이스의 중심 개체를 결정하게 됩니다.

2) 같은 이름이지만 다른 역할을 하는 캐릭터 개체

두 게임 모두 캐릭터 개체가 존재하지만, 역할이 완전히 다릅니다. 메이플스토리의 캐릭터는 플레이어가 만들어가는 데이터로, 계정마다 수가 다르고 레벨과 메소가 계속 실시간으로 변합니다. 슬더스2의 캐릭터는 게임에 미리 정해진 5명의 데이터로, 모든 플레이어가 같은 캐릭터를 공유하며 원본 데이터 값이 변하지 않습니다.

3) 관계 종류와 키 설계의 차이

메이플스토리는 모든 자식 개체가 자신만의 구별 번호를 가지므로 모두 비식별 관계입니다. 슬더스2는 게임 내 얻은 유물만 식별 관계인데, 같은 유물은 중복 보유할 수 없다는 게임 규칙 때문에 부모의 키 조합만으로 행을 구별할 수 있기 때문입니다.

4) 과금 데이터 구조의 유무

메이플스토리는 캐시 충전과 구매가 게임 데이터베이스 안에서 이루어지므로 계정에 잔액을, Cash_Trade 저장해야 하며, 현금과 연결된 기록이기 때문에 기록 저장이 필수적입니다. 슬더스2는 게임 구매가 steam에서 이루어지고 게임 내 재화는 등반 안에서만 유효하므로 과금 관련 개체가 필요 없습니다.